ASD PRACTICAL

Complete Step-by-Step Guide

Git + Java JDK + Jenkins + GitHub Integration

Project used: Jenkins Freestyle Project — *My-First-Pipeline*

Build step: Execute Windows batch command

This document collects the setup steps, configuration, commands, verification, and the eight-point practical write-up in one place.

Accuracy note: The steps below reflect the setup and successful build described in the chat. The job ran a Windows batch build step; a separate Jenkinsfile, deployment, and automatic GitHub webhook trigger were not confirmed.

PART A — COMPLETE SETUP AND EXECUTION

Step 1: Verify Git installation

Open Command Prompt (CMD) and run:

```
git --version
```

Observed version: **git version 2.56.0.windows.1**. If CMD says the command is not recognized, install Git for Windows and reopen CMD.

Step 2: Verify Java JDK installation

Run these commands in CMD:

```
javac -version
where javac
```

Observed compiler version: **javac 21.0.12.1**. The machine showed JDK 21 and JDK 25 compiler paths. For this practical, the verified javac command worked.

Step 3: Open Jenkins

Start/open the Jenkins service, then open the browser and visit **http://localhost:8080** if Jenkins is using its default local port. Complete the initial unlock/setup wizard.

The initial administrator password was read from the Jenkins secrets file at:

```
C:\\ProgramData\\Jenkins\\.jenkins\\secrets\\initialAdminPassword
```

Enter that password in the setup screen, complete the setup steps, and create the administrator account. Keep credentials private; do not include passwords in the practical report.

Step 4: Create the Jenkins project

• From the Jenkins dashboard, choose **New Item**.

• Enter the job name: **My-First-Pipeline**.

• Select **Freestyle project** and choose OK.

• Open the job configuration page.

Step 5: Connect the GitHub repository

• In the job configuration, find **Source Code Management**.

• Select **Git**.

• Paste the URL of your own GitHub repository into **Repository URL**.

• Set the branch specifier to ***/main** if the repository's branch is named main.

• If the repository is private, configure credentials in Jenkins; never paste a password or token into the report.

> The exact repository URL was not recorded in this guide, so use the same URL that you entered in Jenkins.

Step 6: Add the Windows batch build step

Under **Build Steps**, choose **Add build step** → **Execute Windows batch command**. Enter these lines:

```
echo ASD Experiment 8
echo GitHub Jenkins Integration Successful
dir
```

Save the configuration. This command block is stored directly in the Jenkins job configuration; it is not a separate .bat file or Jenkinsfile.

Step 7: Run the build

• Open the **My-First-Pipeline** job.

• Click **Build Now**.

• Open the build number in Build History.

• Choose **Console Output**.

• Check for the two echo messages, a directory listing from dir, and the final result **Finished: SUCCESS**.

Step 8: Understand the result

The successful result means Jenkins checked out the configured repository and completed the batch build step without a reported failure. It does not prove that an application was deployed. A webhook-based automatic trigger was not confirmed; the build was run using Build Now.

PART B — ARCHITECTURE / FLOW

1. Developer	2. GitHub	3. Jenkins SCM	4. Batch Step	5. Console
Code changes	Repository	Checkout files	Run commands	Build status

Flow in words: Developer → GitHub repository → Jenkins repository checkout → Windows batch build step → Console Output / SUCCESS status.

PART C — EIGHT-POINT PRACTICAL WRITE-UP

1. AIM

To integrate a GitHub repository with Jenkins and execute a build step through a Jenkins Freestyle Project.

2. TOOLS / REQUIREMENTS

Requirement	Details
Operating System	Windows
Version Control	Git for Windows — 2.56.0.windows.1
Automation Tool	Jenkins
Programming Tool	Java JDK; javac 21.0.12.1 verified
Remote Repository	GitHub
Jenkins Job	My-First-Pipeline (Freestyle Project)

3. ARCHITECTURE / FLOW DIAGRAM

Developer → GitHub → Jenkins SCM checkout → Execute Windows batch command → Console Output.

4. STEPWISE PROCEDURE

1. Install Git and verify it using git --version.
2. Install a Java JDK and verify it using javac -version.
3. Open Jenkins in a browser and complete the initial setup.
4. Create a Freestyle Project named My-First-Pipeline.
5. Configure Git under Source Code Management and enter the GitHub repository URL.
6. Set the branch specifier to */main if the repository uses main.
7. Add the Windows batch build step and enter the script below.
8. Save the job, click Build Now, and open Console Output.
9. Verify the printed messages and SUCCESS status.

5. SCRIPT / CODE

Filename: Inline Windows batch build step (entered in Jenkins; not saved as a separate file).

```
echo ASD Experiment 8
echo GitHub Jenkins Integration Successful
dir
```

6. COMMANDS EXECUTED

```
git --version
javac -version
where javac
```

Jenkins batch commands:

```
echo ASD Experiment 8
echo GitHub Jenkins Integration Successful
dir
```

Jenkins UI sequence: New Item → My-First-Pipeline → Freestyle project → Configure → Source Code Management: Git → Build Steps: Execute Windows batch command → Save → Build Now → Console Output.

7. EXPECTED OUTPUT & EXPLANATION

```
ASD Experiment 8
GitHub Jenkins Integration Successful
[Directory listing]
Finished: SUCCESS
```

Verification	Result
Git	git version 2.56.0.windows.1
Java compiler	javac 21.0.12.1
Repository checkout	Successful as reported during setup
Batch step	Messages and directory listing displayed
Build result	SUCCESS
Jenkins local port	8080, if default port is used

Explanation: The SUCCESS status indicates that the Jenkins job completed without a reported build-step error. Deployment and webhook-triggered execution were not verified.

8. CONCLUSION

GitHub was connected to a Jenkins Freestyle Project, and the Windows batch build step ran successfully. The console output verified the basic repository checkout and build automation workflow.

QUICK REVISION BEFORE PRACTICAL

• Git command: git --version

• Java compiler command: javac -version

• Jenkins local URL: http://localhost:8080 (if using default settings)

• Job name: My-First-Pipeline

• Branch specifier: */main

• Build step type: Execute Windows batch command

• Key output: GitHub Jenkins Integration Successful

• Final status: Finished: SUCCESS

• Important distinction: current job is Freestyle, so the inline commands are not a Jenkinsfile.